



AI SECURITY

10 Core Skills for Defending LLMs and AI Agents

Maisam Belal Shaikh Ibrahim

Why AI Security?

As LLMs move from chatbots to autonomous agents, they create an entirely new attack surface.



A New Attack Surface

Agents that call tools, browse the web, and take real actions can be manipulated through prompts, documents, or tool results — not just malicious code.



Traditional Security Doesn't Cover It

Firewalls and antivirus don't stop jailbreaks, prompt injection, or a model leaking its own instructions or private data.



Layered Defense Is Essential

No single control catches everything — real AI security combines input filtering, output validation, access control, and continuous testing.

The 10 Skills at a Glance





SKILL 01

Automated PII Masking & Redaction

WHAT IT IS

Automatically detects and masks personal data — emails, phone numbers, SSNs, credit cards — before it reaches a user, a log file, or storage.

WHY WE NEED IT

LLM outputs can surface sensitive data from earlier context or retrieved documents. Without redaction, that data can leak into logs or reach people who shouldn't see it — creating privacy and compliance risk.

TOOLS & TECHNOLOGIES

● Microsoft Presidio

● spaCy NLP

● LLM Guard

● Regex-based scanners

Microsoft Presidio



WHAT IT IS

An open-source PII detection and anonymization engine from Microsoft. It uses NLP models to recognize entities like names, emails, phone numbers, and ID numbers inside free text, then lets you mask, redact, or replace them.

HOW IT'S USED

- 1 Install the analyzer and anonymizer packages via pip
- 2 Pass text into the AnalyzerEngine to detect PII entities
- 3 Feed the detected entities into the AnonymizerEngine to mask or replace them
- 4 Return the sanitized text before it reaches a user, log, or database

WHAT IT IS

Pauses an AI agent before it executes a high-impact action (sending an email, deleting data) and requires explicit human approval first.

WHY WE NEED IT

Agents reason over untrusted input, so a manipulated agent could take irreversible real-world actions. A human approval gate is a circuit breaker — even a compromised agent can't act without sign-off.

TOOLS & TECHNOLOGIES

- NeMo Guardrails
- Workflow/notification APIs (Slack, email)
- Custom approval services



SKILL 02

Context-Aware Human-in-the-Loop Gatekeeping

NVIDIA NeMo Guardrails



WHAT IT IS

An open-source framework for wrapping an LLM application in programmable 'rails' — input, output, and dialogue rules — defined through a simple config file plus a flow language called Colang.

HOW IT'S USED

- 1 Install the nemoguardrails package via pip
- 2 Define input/output/dialogue rules in a config.yml file
- 3 Write Colang flows describing what to block or require approval for
- 4 Route every model call through the Rails engine before it executes



SKILL 03

Defending Against Indirect Prompt Injection

WHAT IT IS

Tests and defends against instructions hidden inside content a model retrieves — a document, webpage, or email — rather than in the user's own message.

WHY WE NEED IT

In RAG pipelines and browsing agents, an attacker never needs direct access to the system — planting a hidden instruction in content the model will read is enough to attempt a hijack.

TOOLS & TECHNOLOGIES

- LLM Guard (PromptInjection scanner)

- garak

- RAG input sanitizers

LLM Guard — PromptInjection Scanner



WHAT IT IS

One of LLM Guard's many input scanners, purpose-built to detect both direct and indirect prompt injection attempts in text before it reaches the model, returning a safety verdict and a risk score.

HOW IT'S USED

- 1 Install the llm-guard package via pip
- 2 Import the PromptInjection scanner from input_scanners
- 3 Call `scan_prompt()` on incoming text (including retrieved documents)
- 4 Block or forward the request based on the returned verdict and score

WHAT IT IS

Explicitly allowlists every tool an AI agent may use, and the exact argument shapes it may call them with — anything not listed is automatically rejected.

WHY WE NEED IT

As agents gain more tools over time, it becomes easy to lose track of what they can actually do. Denying by default means only pre-approved actions are ever possible in the first place.

TOOLS & TECHNOLOGIES

- JSON Schema validation
- Tool/capability registries
- Least-privilege IAM



SKILL 04

Deny-by-Default Capability Mapping

JSON Schema (jsonschema library)



WHAT IT IS

A Python library implementing the JSON Schema standard, used to define the exact expected shape of a tool call's arguments — so anything malformed or unexpected is rejected automatically, before it ever executes.

HOW IT'S USED

- 1 Install the jsonschema package via pip
- 2 Define a schema describing each tool's valid arguments
- 3 Call `validate()` on incoming tool-call arguments against that schema
- 4 Reject the call immediately if validation fails



SKILL 05

Outbound Response Validation & Safety Filters

WHAT IT IS

Scans an LLM's generated response before it reaches the user — checking for toxic content, leaked sensitive data, or policy violations.

WHY WE NEED IT

A model can occasionally produce an unsafe or non-compliant response even with good training. An independent output-side check catches this without relying solely on the model's own judgment.

TOOLS & TECHNOLOGIES

● Llama Guard 3

● NeMo Guardrails

● LLM Guard output scanners

Llama Guard 3



WHAT IT IS

Meta's safety-classification model, built on Llama 3.1 and fine-tuned specifically to judge whether a prompt or response is safe — returning a verdict plus the exact policy category violated, if any.

HOW IT'S USED

- 1 Download the model via Hugging Face (requires accepting Meta's license)
- 2 Format the prompt or response using the provided chat template
- 3 Run the model to generate a safe/unsafe verdict and violation category
- 4 Block or allow the content downstream based on that verdict

WHAT IT IS

Tests whether an LLM's hidden system prompt — its role, restrictions, and internal configuration — can be extracted through direct or indirect prompting.

WHY WE NEED IT

A leaked system prompt gives an attacker a roadmap to bypass a model's restrictions, and may expose confidential business logic embedded in the instructions.

TOOLS & TECHNOLOGIES

● Prompt-leak test suites

● garak

● Custom leak detectors



SKILL 06

Post-Inference System Prompt Protection

garak (Leak-Testing Probes)



WHAT IT IS

NVIDIA's open-source LLM vulnerability scanner includes probes specifically designed to test whether a model leaks its system prompt or training data, in addition to its jailbreak-testing capabilities.

HOW IT'S USED

- 1 Install garak via pip
- 2 Point it at your target model (API, local model, etc.)
- 3 Select a leak-testing probe, such as `promptinject` or `leakreplay`
- 4 Review the generated report for any successful leak attempts



SKILL 07

Red-Teaming LLMs (Adversarial Testing)

WHAT IT IS

Systematically probes an LLM with large batteries of known jailbreak and adversarial prompts to discover weaknesses in its safety behavior.

WHY WE NEED IT

Manually testing one jailbreak at a time doesn't scale and misses techniques you haven't thought of. Automated red-teaming turns safety testing into a repeatable, measurable process.

TOOLS & TECHNOLOGIES

- garak (NVIDIA)

- PyRIT (Microsoft)

- Adversarial prompt libraries

garak (NVIDIA)



WHAT IT IS

An open-source LLM vulnerability scanner, often described as 'nmap for LLMs' — it runs large, organized batteries of adversarial prompts against a target model and scores whether each attack succeeded.

HOW IT'S USED

- 1 Install garak via pip and point it at your target model
- 2 Choose a probe category, e.g. `dan.DAN_Jailbreak` for persona-override jailbreaks
- 3 Run the scan — garak sends every prompt variation automatically
- 4 Review the pass/fail report per prompt to see where defenses held or broke

WHAT IT IS

The full layered control set governing the moment an agent decides which tool to call: allowlisting, argument validation, scoped credentials, approval gates, and audit logging.

WHY WE NEED IT

This is the highest-risk point in an agentic system — a single successful manipulation here can turn an agent into a 'confused deputy' taking real, harmful actions.

TOOLS & TECHNOLOGIES

- NeMo Guardrails
- JSON Schema
- AWS STS / scoped IAM
- Audit logging / SIEM



SKILL 08

Securing Agentic AI Tool Invocation

NeMo Guardrails — Tool Rails



WHAT IT IS

The same NeMo Guardrails framework from Skill 2, applied here to gate tool and plugin calls specifically — wrapping an agent's available tools so every call passes through policy rails before it executes.

HOW IT'S USED

- 1 Register each tool's name and allowed arguments in the rails config
- 2 Define an approval flow for high-impact tools
- 3 Route every agent tool-call attempt through the rails engine
- 4 Log the resulting decision — allow, deny, or require-approval — for each call



SKILL 09

Structural Error Masking & Exception Management

WHAT IT IS

Ensures that any internal failure inside a security-critical function never leaks internal details, and always defaults to a safe denial rather than an accidental allow.

WHY WE NEED IT

If a bug or unexpected input crashes a policy-enforcement function, the worst outcome is for that ambiguity to be treated as permission. Failing closed guarantees a bug can never grant unauthorized access.

TOOLS & TECHNOLOGIES

- Structured try/except policy wrappers
- Centralized logging
- SIEM alerting

Python Exception Handling (try / except)



WHAT IT IS

Not a third-party product, but a core language feature — used here as the structural guarantee that any unexpected failure inside a security-critical function resolves to a safe default instead of crashing or leaking details.

HOW IT'S USED

- 1 Wrap the security-critical decision logic in a try block
- 2 Catch broad exceptions with a general except Exception clause
- 3 Return a generic, safe 'deny' decision from inside the except block
- 4 Log the real error internally, without exposing it to the caller

WHAT IT IS

Caps how much a single user or request can demand from an LLM system — limiting request volume over time and capping the length of any single prompt.

WHY WE NEED IT

Without limits, a single actor can drive up cost, degrade service for others, or use high-volume systematic querying as a building block for model-extraction attacks.

TOOLS & TECHNOLOGIES

- API gateways
- LLM Guard TokenLimit scanner
- Rate-limiter middleware



SKILL 10

User Rate Limiting & Token Throttling

LLM Guard — TokenLimit Scanner



WHAT IT IS

One of LLM Guard's input scanners, purpose-built to reject prompts that exceed a configured token or length limit before they're ever passed to the model — a simple, deterministic guard against oversized requests.

HOW IT'S USED

- 1 Install the llm-guard package via pip
- 2 Import TokenLimit from input_scanners and set a maximum token count
- 3 Include it in your input scanner pipeline alongside other checks
- 4 Reject any prompt that exceeds the limit before further processing



Defense in Depth

No single control secures an AI system. Input filtering, output validation, access control, human oversight, and continuous adversarial testing work together — each catching what the others might miss.



These 10 skills form a practical foundation for securing modern LLM and agentic AI systems.